
Assignment for ASI :Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles Lakshminarayanan et al. (2017)

Chaoui Kouraichi Ahmed Taha

Abstract

This report presents a summary and partial reproduction of *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles* by Lakshminarayanan et al. (2017). The paper proposes a non-Bayesian alternative for predictive uncertainty estimation based on three ingredients: (1) training probabilistic neural networks with proper scoring rules, (2) adversarial training for smoothing predictions, and (3) ensembling independently trained networks. We summarize the key ideas, reproduce the toy regression experiment, and discuss the strengths and limitations of the approach. The code is available at: <https://github.com/chaoui-ahmed/ASI-PAPER/>.

1 Introduction

Reliable uncertainty estimation is critical for deploying machine learning models in safety-sensitive applications such as medical diagnosis, autonomous driving, and scientific discovery. A model should not only be accurate but also know what it does not know: it should express high confidence on familiar inputs and high uncertainty on out-of-distribution data.

The dominant paradigm for uncertainty in neural networks is Bayesian deep learning, where one places a prior over the weights θ and computes the posterior $p(\theta \mid \mathcal{D})$. Exact Bayesian inference is intractable for neural networks due to high-dimensional, non-convex, multimodal posteriors. Approximate methods include variational inference (VI), which fits a tractable family $q(\theta; \nu)$ by maximizing the ELBO; MCMC methods such as Metropolis-Hastings or Hamiltonian Monte Carlo; the Laplace approximation, which fits a Gaussian at the MAP estimate; and MC-Dropout, which reinterprets dropout at test time as approximate variational inference.

Lakshminarayanan et al. propose deep ensembles as a simpler, scalable alternative that requires minimal modifications to standard neural network training yet provides strong uncertainty estimates.

2 Summary of the Paper

2.1 Problem Setup

Given a training set $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, the goal is to learn a neural network that outputs a *predictive distribution* $p_\theta(y \mid \mathbf{x})$ rather than a point estimate. For regression, the network outputs both a mean $\mu_\theta(\mathbf{x})$ and a variance $\sigma_\theta^2(\mathbf{x}) > 0$, modeling a heteroscedastic Gaussian:

$$p_\theta(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mu_\theta(\mathbf{x}), \sigma_\theta^2(\mathbf{x})). \quad (1)$$

For classification with K classes, $p_\theta(y \mid \mathbf{x})$ is a categorical distribution obtained via softmax.

2.2 Proper Scoring Rules

A **proper scoring rule** $S(p_\theta, (y, \mathbf{x}))$ assigns a numerical score to a predictive distribution, rewarding calibration. It is proper if $S(p_\theta, q) \leq S(q, q)$ with equality iff $p_\theta = q$. This ensures that maximizing the score finds the exact true distribution.

The negative log-likelihood (NLL) is a proper scoring rule. For the Gaussian output in Eq. (1), the training loss becomes:

$$\mathcal{L}(\theta) = \sum_{n=1}^N \left[\frac{\log \sigma_\theta^2(\mathbf{x}_n)}{2} + \frac{(y_n - \mu_\theta(\mathbf{x}_n))^2}{2\sigma_\theta^2(\mathbf{x}_n)} \right]. \quad (2)$$

2.3 Adversarial Training for Smoothing

The authors add small, intentional changes to the inputs to make the predictions smoother:

$$\mathbf{x}'_n = \mathbf{x}_n + \epsilon \text{sign}(\nabla_{\mathbf{x}_n} \mathcal{L}(\theta, \mathbf{x}_n, y_n)), \quad (3)$$

where ϵ is set to 1% of the input range per dimension. The new loss $\mathcal{L}(\theta, \mathbf{x}_n, y_n) + \mathcal{L}(\theta, \mathbf{x}'_n, y_n)$ forces the network to give the same answer even if the input changes slightly. This makes the model more stable.

2.4 Ensemble Combination

An ensemble of M networks $\{\theta_m\}_{m=1}^M$, each trained independently from different random initializations on the full dataset, creates an average of all their predictions:

$$p(y | \mathbf{x}) = \frac{1}{M} \sum_{m=1}^M p_{\theta_m}(y | \mathbf{x}). \quad (4)$$

For regression, this mixture of Gaussians is approximated by a single Gaussian with:

$$\mu_*(\mathbf{x}) = \frac{1}{M} \sum_m \mu_{\theta_m}(\mathbf{x}), \quad \sigma_*^2(\mathbf{x}) = \frac{1}{M} \sum_m [\sigma_{\theta_m}^2(\mathbf{x}) + \mu_{\theta_m}^2(\mathbf{x})] - \mu_*^2(\mathbf{x}). \quad (5)$$

The total predictive variance naturally splits into:

- **Aleatoric uncertainty:** $\frac{1}{M} \sum_m \sigma_{\theta_m}^2(\mathbf{x})$ — the average predicted noise, which cannot be reduced even with infinite data.
- **Epistemic uncertainty:** $\frac{1}{M} \sum_m \mu_{\theta_m}^2(\mathbf{x}) - \mu_*^2(\mathbf{x})$ — the variance of the means, which shows how much the models disagree with each other.

2.5 Why Ensembles Work: Connection to Bayesian Inference

Neural networks have many different good solutions. Different random starting weights lead networks to find completely different solutions. While Bayesian methods like VI often just look around one single solution $q(\theta; \nu)$, deep ensembles naturally capture multiple different ones.

Dropout works like a basic Bayesian method with $q(\theta) = (1 - p)\mathcal{N}(m, \sigma^2) + p\mathcal{N}(0, \sigma^2)$. However, because all dropout masks share the exact same core weights, MC-Dropout only explores a very small area. Deep ensembles, by contrast, explore entirely different areas, making them much better at detecting unknown data.

Furthermore, theory shows that very wide random networks act like Gaussian processes. This proves that random starting weights naturally create good diversity: each network is acting like a sample from a different part of the function space.

2.6 Real-World Regression Benchmarks

Beyond the toy experiment, the authors evaluate deep ensembles on 10 UCI regression datasets (Boston Housing, Concrete, Energy, Kin8nm, Naval, Power Plant, Wine, Yacht, Protein, Song). On

each dataset, ensembles are compared against Probabilistic Backpropagation (PBP) and MC-Dropout using test RMSE and test log-likelihood. Deep ensembles consistently match or outperform both baselines: they achieve the best test log-likelihood on 6 out of 10 datasets and competitive RMSE everywhere. The improvement is most pronounced on larger datasets (Protein, Song), where the diversity from independent training is most beneficial.

2.7 Classification Experiments

For classification, the authors evaluate on MNIST, SVHN, and CIFAR-10 using ensembles of $M = 5$ networks. Both adversarial training and ensembling independently reduce classification error and Brier score. Combining both ingredients yields the best results: on SVHN, the ensemble with adversarial training reduces the Brier score by approximately 20% compared to a single network. The Brier score penalizes both miscalibration and lack of sharpness, making it a more informative metric than accuracy alone.

For out-of-distribution (OOD) detection, the authors train on one dataset (e.g., MNIST) and evaluate predictive entropy on a different dataset (NotMNIST). Deep ensembles assign significantly higher entropy to OOD samples compared to MC-Dropout, which tends to make overconfident predictions even on unknown data.

2.8 Accuracy as a Function of Confidence

A particularly practical result is the analysis of prediction accuracy as a function of confidence threshold. The authors progressively filter out predictions where the model’s confidence falls below a threshold, and measure accuracy on the remaining “trusted” predictions. Deep ensembles show a desirable property: as the confidence threshold increases, accuracy on retained predictions improves sharply, approaching near-perfect accuracy at high thresholds. MC-Dropout, by contrast, still makes confident errors — it assigns high confidence to incorrect predictions, making it unreliable as a trust filter. This has direct implications for safety-critical deployments where a model should abstain rather than make uncertain predictions.

3 Reproduced Results

3.1 Toy Regression Experiment

We reproduce the 1D toy regression experiment from Section 3.2 of the paper. The dataset consists of 20 training points drawn from $y = x^3 + \epsilon$ where $\epsilon \sim \mathcal{N}(0, 9)$, following the setup of Hernández-Lobato and Adams (2015).

Implementation details. We implement the experiment in Python using JAX. Each network in the ensemble is a fully connected MLP with one hidden layer of 50 units and ReLU activations. The final layer outputs two values: $\mu_\theta(\mathbf{x})$ and $\log \sigma_\theta^2(\mathbf{x})$. We train using the NLL loss from Eq. (2) with Adam optimizer (learning rate 0.01) for 5000 epochs.

Experimental protocol.

1. **Single NN with MSE:** Train a single network to minimize MSE. No variance is learned; uncertainty is unavailable.
2. **Single NN with NLL:** Train a single network to minimize NLL (Eq. (2)). The network learns both μ and σ^2 .
3. **Ensemble of $M = 5$ NNs with NLL:** Train 5 networks independently from random initializations; combine using Eq. (5).

Results. Our reproduction confirms the paper’s key findings:

- A single NN trained with MSE provides only a point prediction with no uncertainty estimate.
- A single NN trained with NLL learns a reasonable variance but becomes overconfident far from the training data — the uncertainty does not grow enough in areas it hasn’t seen.

- The 5-network ensemble works perfectly: the predictive variance is small near the training data and grows as we move away, showing that the model is correctly less certain in unexplored areas.

Figure 1 shows the three experimental protocols side by side. The growing uncertainty band in the ensemble plot (right) is the key result: the model correctly expresses higher uncertainty in regions where it has not seen data.

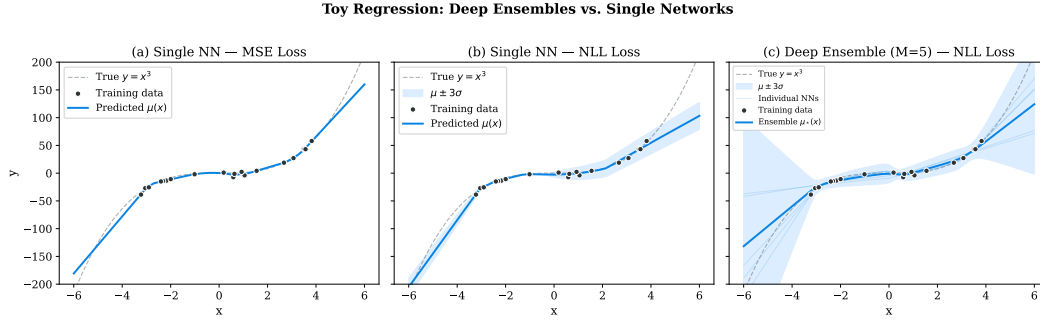


Figure 1: Toy regression experiment ($y = x^3 + \epsilon$, 20 training points). (a) Single NN with MSE provides only a point prediction. (b) Single NN with NLL learns variance but remains overconfident far from the data. (c) Deep ensemble ($M = 5$) with NLL produces well-calibrated uncertainty that grows away from training data. Shaded regions show $\mu \pm 3\sigma$.

Figure 2 shows the uncertainty decomposition. The aleatoric component (green) remains roughly constant — reflecting the irreducible noise $\epsilon \sim \mathcal{N}(0, 9)$ — while the epistemic component (red) grows sharply outside the training range $[-4, 4]$, reflecting model disagreement in unexplored regions.

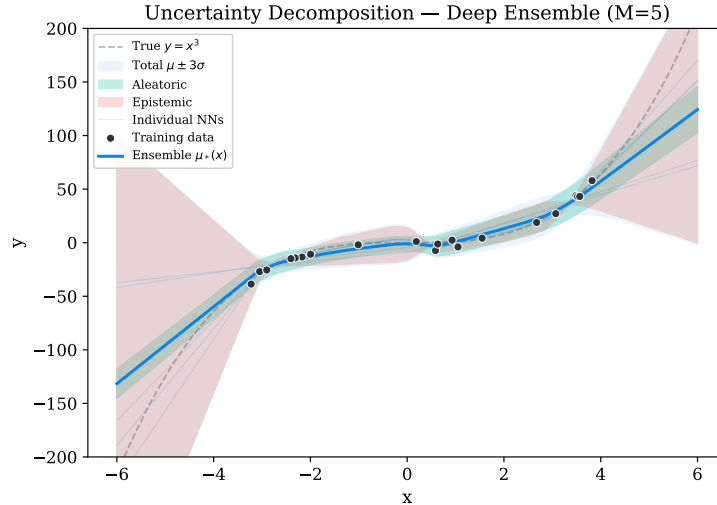


Figure 2: Uncertainty decomposition for the deep ensemble. Aleatoric uncertainty (green) captures irreducible data noise; epistemic uncertainty (red) captures model disagreement and grows away from training data.

The predictive variance grows away from training data. The main advantage of deep ensembles is getting this result using standard neural networks without doing complex Bayesian math.

3.2 Comparison with Bayesian Approaches

Deep ensembles combine fast scaling, the ability to find multiple solutions, and easy implementation. Unlike VI or Laplace, which fit a single curve, ensembles capture many. Unlike MCMC, they are

Table 1: Comparison of uncertainty methods studied.

Method	Scalability	Multimodal	Simplicity
Exact Bayes	Low	Yes	Medium
VI / Mean-field	High	No	Medium
MCMC	Low	Yes	Low
Laplace	High	No	Medium
MC-Dropout	High	No	High
Deep Ensembles	High	Yes	High

completely parallel and do not need complex checks to see if they finished training. The main cost is that training M networks takes M times the memory and computing power.

4 Discussion

4.1 Strengths of the Approach

Simplicity and practicality. The method needs only three changes to standard training: (1) change the output layer to predict mean and variance, (2) use NLL loss, and (3) train M copies. We don’t need to choose complex math distributions, calculate the ELBO equation, or run convergence checks.

Scalability. Training is completely parallel. The authors demonstrate results on ImageNet with Inception networks — a scale that is too difficult for most Bayesian methods. This solves a major problem of MCMC and VI methods.

Strong OOD detection. The experiments on MNIST vs. NotMNIST and SVHN vs. CIFAR-10 show that deep ensembles produce higher uncertainty on unknown classes compared to MC-Dropout, which tends to be overconfident. Showing high uncertainty on unknown data is exactly what we need for active learning.

4.2 Limitations

No strict mathematical prior. Unlike Bayesian methods, deep ensembles have no explicit prior $p(\theta)$. The implicit “prior” comes purely from random initialization and the network structure. Weight decay acts like a Gaussian prior, but ensembles do not use this to measure uncertainty.

Diversity is not guaranteed. The method hopes that random initialization produces diverse solutions. Neural networks have many good solutions, but there is no mathematical guarantee that random starting weights will cover all of them well.

Computational cost. Training and storing M networks (typically $M = 5$) takes much more computing power. For comparison, Gaussian processes calculate uncertainty directly, though GPs get extremely slow if the dataset is large. The authors propose knowledge distillation as a solution: once an ensemble is trained, its combined predictions can be used to train a single “student” network that mimics the ensemble’s output distribution. This would reduce inference cost from $M \times$ to $1 \times$ while retaining much of the uncertainty quality.

Uncertainty calibration. While the paper shows good calibration results, using proper scoring rules does not guarantee perfect calibration if you don’t have enough data. The connection between proper scoring rules and Bayesian math suggests that the NLL loss has a natural Bayesian meaning, which partly explains why it works so well in practice.

4.3 Last section

Deep ensembles are frequentist but act like Bayesian models — growing uncertainty away from data, OOD detection. Theory provides some proof: if each random network acts like a sample from a Gaussian Process, then averaging them works a lot like Bayesian math.

For continual learning, deep ensembles could be used as a starting point for new tasks, though it lacks the clean, strict math updates $p(\theta \mid \mathcal{D}_{1:T}) \propto p(\mathcal{D}_T \mid \theta) p(\theta \mid \mathcal{D}_{1:T-1})$ of true Bayesian methods.

A Implementation Details

The code is implemented in Python using JAX. Here are some key implementation choices:

- **Framework:** JAX with `jax.numpy` for automatic differentiation.
- **Architecture:** Single hidden layer MLP with 50 units and ReLU activation; two output heads for μ and $\log \sigma^2$.
- **Variance parameterization:** We output $\log \sigma^2$ and apply softplus: $\sigma^2 = \log(1 + \exp(\cdot)) + 10^{-6}$, following the paper’s recommendation.
- **Optimizer:** Adam with learning rate 0.01, batch size equal to full dataset (20 points).
- **Ensemble:** $M = 5$ networks, each initialized with different JAX PRNG keys.
- **Prediction:** Combined via Eq. (5).

The NLL loss function directly mirrors the Gaussian log-likelihood